

---

# From Interfaces to an Executable SC/DPS Plan

A simulation and verification framework that turns system interactions into the development schedule

**Manuel Arroyave**

SC/DPS coordination discussion — 29 July 2026

**DUNE**  
DEEP UNDERGROUND  
NEUTRINO EXPERIMENT

 **Fermilab**

# This proposal connects existing work

---

## INTEGRATION, NOT A RESET

### What the collaboration already has

- ▶ Years of subsystem design, firmware, controls, and test-stand work.
- ▶ Working protocols, serializers, device servers, ICDs, and DAQ packages.
- ▶ Experienced consortium and integration teams who know their hardware.
- ▶ Existing SC and DPS decisions that must remain traceable.

### The missing shared artifact

- ▶ One lifecycle-aware view of interactions across subsystem boundaries.
- ▶ Explicit ownership of normal control, protection, failure, and recovery.
- ▶ An executable link from interface requirements to package-level tests.
- ▶ A development schedule derived from real dependencies and risks.

**Reuse the subsystem work. Connect it into one versioned, testable system contract.**

# Why move now?

## INTERFACE ASSUMPTIONS ARE BECOMING PHYSICAL CONSTRAINTS

### 1 Design

Protocols, power architecture, authority, states, timing.

### 2 Procurement

PLCs, PDUs, servers, switches, UPS, field interfaces.

### 3 Installation

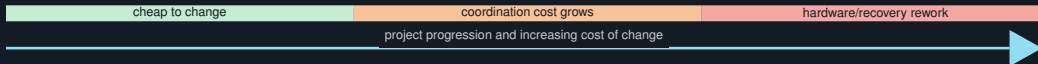
Cabling, rack maps, network zones, interlock paths.

### 4 Commissioning

Recovery, partial states, fault response, acceptance evidence.

### 5 Operation

Availability, change control, incident response.



## Move fast on unknowns

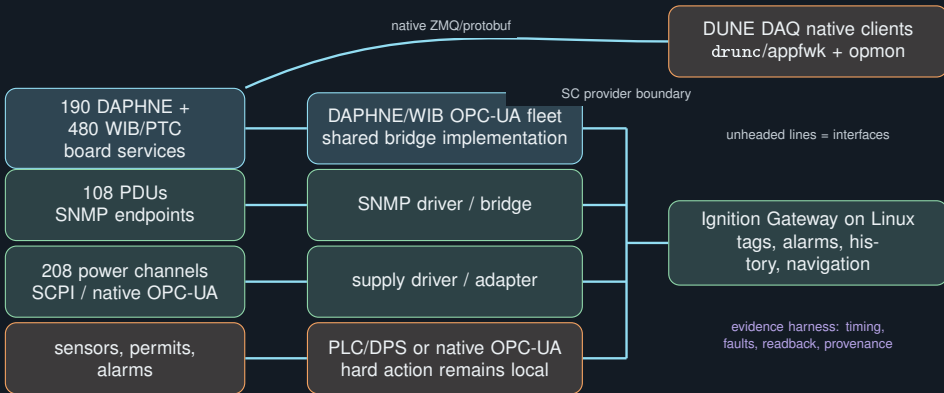
Resolve ownership, state constraints, sequencing, and independent protection before they are frozen into hardware and deployment choices.

## Do not rush protection

Speed means earlier questions and executable evidence—not bypassing safety, consortium, electrical, or configuration reviews.

# A draft bench already exists

## SEPARATE NATIVE PATHS; ONE OPERATOR AND TEST VIEW



Current synthetic inventory: 1,818 logical objects • 40,755 published variables • 986 writable endpoints • one shared Ignition operator layer.

# Evidence must remain labelled

## WHAT WAS MEASURED, WHAT WAS PROJECTED, WHAT IS STILL OPEN

### Tested locally

- ▶ 12,000 balanced configure/readback trials; zero observed errors.
- ▶ One 40,755-variable OPC-UA snapshot in 0.516 s.
- ▶ One local pass over 986 writable endpoints in 0.517 s.
- ▶ Four same-host OPC-UA processes at the measured monitoring knee.

T = workstation evidence

### Source-derived / projected

- ▶ DAPHNE explicit-wait floor: 2.6901 s per board.
- ▶ Four-FEMB WIB explicit-wait floor: 16.01–16.10 s.
- ▶ `drunc/appfwk` fan-out and queue models.
- ▶ Bridge/leaf sizing with posterior timeout and retry uncertainty.

P = planning model

### Not yet demonstrated

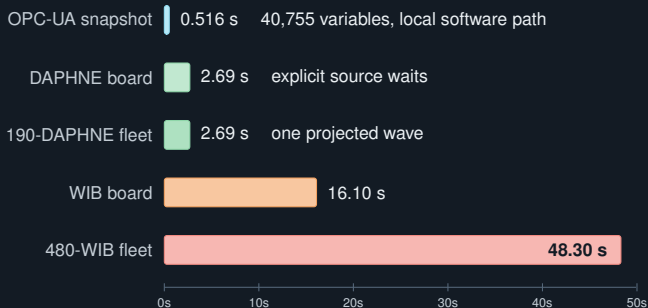
- ▶ Real PLCs, switches, detector networks, and production endpoints.
- ▶ Hardware I/O and calibration time beyond explicit source waits.
- ▶ Full Ignition subscription, historian, alarm, and client load.
- ▶ Safety integrity, physical interlocks, failover, or production sizing.

U = owner input / test needed

The bench is useful because it separates these categories instead of turning a local result into a detector guarantee.

# Monitoring is not the sizing workload

## CONFIGURATION, QUEUES, AND RECOVERY SET THE CRITICAL PATH



Fleet rows are no-failure floors for four shared bridge queues with 48 active leaves each: 47–48 DAPHNEs/bridge form one 2.69 s wave; 120 WIBs/bridge form three 16.10 s waves. Both are projections, not hardware measurements.

### Why configuration is heavier

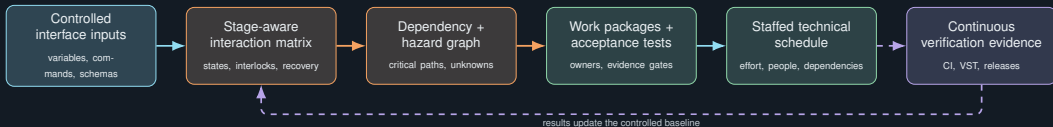
- ▶ Explicit firmware calibration and verification waits.
- ▶ DAQ controller/application nesting and bounded fan-out.
- ▶ Per-bridge queues, timeout, retry, and independent readback.
- ▶ State, power, and timing prerequisites before a command is legal.

### Planning point

1 Ignition Gateway  
4 DAPHNE/WIB bridge instances  
47–48 DAPHNE + 120 WIB/bridge  
48 active leaves/bridge

# The proposal in one line

## THE MATRIX PRODUCES THE SCHEDULE



**The development schedule is an output of the interaction matrix—  
not a calendar assembled from independent subsystem task lists.**

The matrix sets priority and order. Effort estimates, staffing, procurement, and review capacity set dates.

# What must be captured in the matrix?

---

## ENOUGH INFORMATION TO TURN EACH INTERACTION INTO WORK AND EVIDENCE

### Identity and authority

- ▶ Source and destination.
- ▶ Variable or command; type, unit, range, rate.
- ▶ Command owner and permitted actor.
- ▶ Applicable lifecycle stage.
- ▶ Source ICD/schema and controlled revision.

### Constraints and response

- ▶ Preconditions and allowed detector states.
- ▶ Permits, inhibits, and interlocks.
- ▶ Sequence, deadline, timeout, stale limit.
- ▶ Expected response and independent readback.
- ▶ Failure, fallback, retry, and safe response.

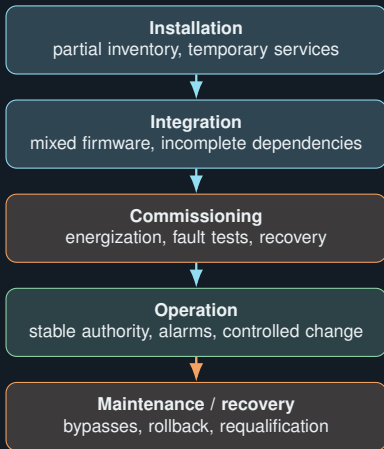
### Closure and evidence

- ▶ Recovery, reset, rollback, and latching.
- ▶ Responsible subsystem and integration owners.
- ▶ Acceptance test and quantitative criterion.
- ▶ Simulator, VST, hardware, or proof evidence.
- ▶ Explicit unknown, conflict, and due date.

Every unresolved cell is visible schedule risk; every closed row can become a reviewed work package and regression test.

# Stage and state both matter

## THE CONSTRAINTS ARE NOT ONLY A FUNCTION OF CRYOGENICS



## State axes that change an allowed action

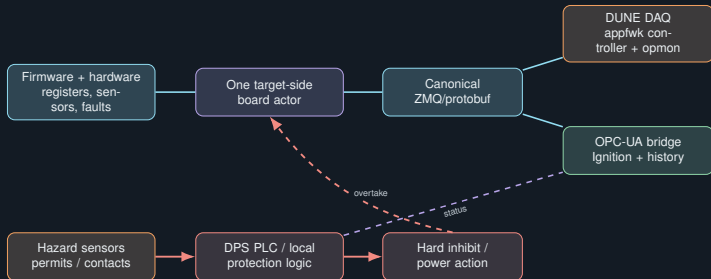
|                   |                                                 |
|-------------------|-------------------------------------------------|
| <b>Cryogenics</b> | warm, cool-down, stable, abnormal               |
| <b>Power</b>      | absent, partial, ramping, stable, trip          |
| <b>DAQ/timing</b> | boot, initialised, configured, running, fault   |
| <b>Network</b>    | healthy, degraded, partitioned, stale           |
| <b>Authority</b>  | local, commissioning, SC, DAQ, DPS, maintenance |

## Allowed action depends on lifecycle · state · permits · communications · authority

Each consortium must define both normal SC behavior and its DPS/failure contract at these boundaries. Nominal variables alone do not define the integration behavior.

# One actor, two clients, independent protection

## THE LOGICAL CONTRACT BEHIND DAPHNE AND WIB INTEGRATION

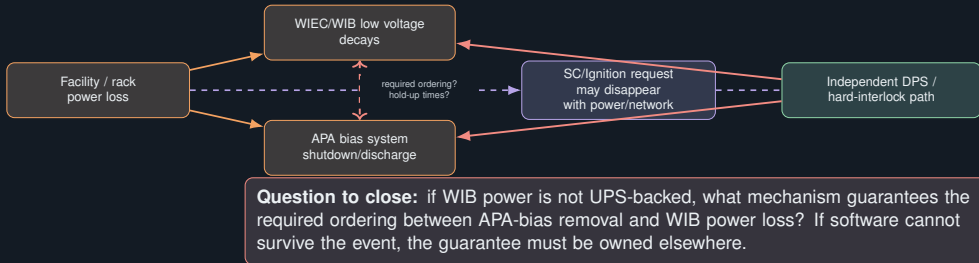


### Non-negotiable rules

- ▶ One process owns AXI, I2C, SPI, DAC, reset, and power transactions.
- ▶ DAQ and SC may read the same cached status in parallel.
- ▶ Every write is serialized, state/authority gated, audited, and read back.
- ▶ `opmonlib` is a DAQ projection, not a second device schema.
- ▶ Required protection must survive the failures for which it is credited.

# Example: power loss exposes a system requirement

A CONDITIONAL QUESTION, NOT A CLAIM THAT THE CURRENT DESIGN IS UNSAFE



## Inputs needed

UPS/hold-up, discharge time, field wiring, PLC/DPS logic, network and power failure modes.

## Matrix output

Requirement, owner, independent path, allowed states, timing bound, recovery rule.

## Schedule output

Design task, integration test, proof test, acceptance gate, unresolved deadline.

# What must be managed—and where

---

## THE CONFIGURATION BASELINE IS LARGER THAN SOURCE CODE

### Controlled in Git

- ▶ Matrix, interface register, ADRs, risks, ownership.
- ▶ Canonical schemas, descriptors, compatibility policy.
- ▶ Firmware/server/DAQ/bridge source and tests.
- ▶ PLC logic and approved deployment exports.
- ▶ Deployment templates, release BOM, rollback.
- ▶ Test definitions and evidence metadata.

### External sources of record

- ▶ EDMS: approved requirements and ICDs.
- ▶ HWDB: installed identity, topology, connections.
- ▶ Artifact registry: firmware, packages, containers, checksums.
- ▶ Secret store: keys, tokens, credentials.
- ▶ Historian/operations DB: runtime samples and alarm state.

### Managed as one release

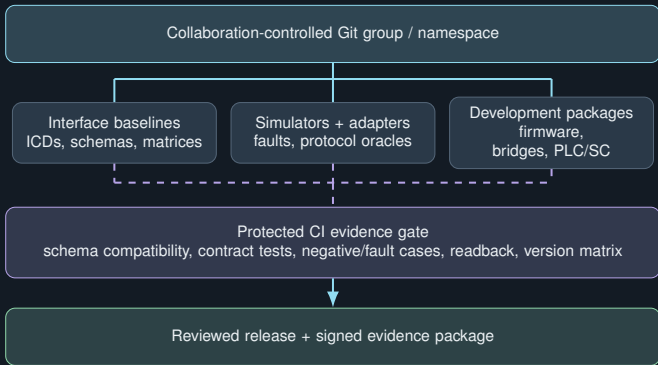
- ▶ Firmware/register ABI.
- ▶ Board service + schema.
- ▶ DUNE DAQ package + opmon mapping.
- ▶ OPC-UA bridge + Ignition export.
- ▶ DPS/permit revision.
- ▶ Configuration/HWDB export.
- ▶ Cross-system acceptance evidence.

#### Rule

One compatible bill of materials pins the exact tested set.

# Make the repositories enforce the contract

## COLLABORATION-OWNED GOVERNANCE + EXECUTABLE COMPATIBILITY

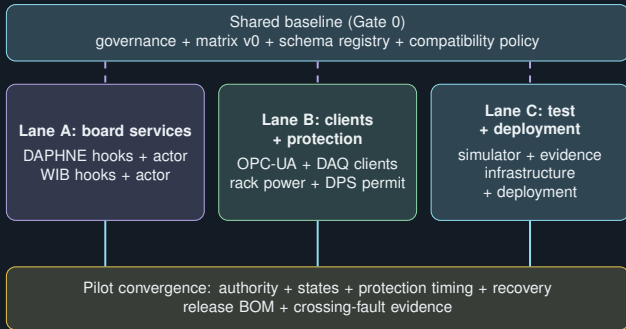


### Controls to agree with DAQ Git admins

- ▶ Named maintainers, CODEOWNERS, interface owners.
- ▶ Protected branches and release rules.
- ▶ Required peer and interface-owner reviews.
- ▶ Versioned schemas and compatibility policy.
- ▶ Reuse DUNE DAQ release and CI conventions.
- ▶ Retained CI evidence and risk/decision records.

# Parallel lanes, explicit convergence gates

INDEPENDENT IMPLEMENTATION WHERE POSSIBLE; SHARED DECISIONS WHERE REQUIRED



## May proceed in parallel

- ▶ Lane A: DAPHNE and WIB hooks and board actors.
- ▶ Lane B: bridge/DAQ clients and rack-power/DPS path.
- ▶ Lane C: simulator, CI evidence, and deployment skeleton.

## Must converge before writes/release

Canonical schema ownership; command authority; allowed states; timing and failure survival; independent readback; recovery; compatible artifact set.

# Begin with two vertical slices

---

SMALL ENOUGH TO CLOSE; BROAD ENOUGH TO EXPOSE SYSTEM DEPENDENCIES

## 1. Shared DAPHNE/WIB OPC-UA path

- ▶ Preserve native ZMQ/protobuf device contracts.
- ▶ Publish both board classes through one fleet bridge implementation.
- ▶ Exercise monitoring, guarded configuration, queueing, and readback.
- ▶ Trace `drunc/appfwk` init and conf dependencies.
- ▶ Measure bridge count, leaf concurrency, latency, error, and recovery.

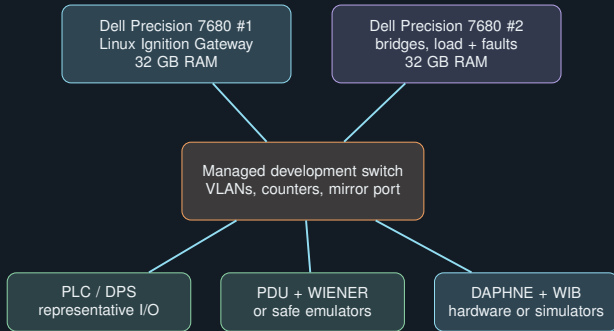
## 2. Rack power and DPS-permit path

- ▶ PDU, WIENER, sensors, PLC/DPS permits, alarms, and latches.
- ▶ Normal energization and controlled shutdown sequence.
- ▶ Network loss, stale data, trip, reset, and recovery.
- ▶ Demonstrate that Ignition cannot bypass the independent permit.
- ▶ Capture owner-approved timing and failure evidence.

**Crossing test:** inject one power, temperature, or communications fault that affects both board configuration and rack protection, then verify the complete recovery path.

# A practical Vertical Slice Test platform

DEVELOPMENT HARDWARE NOW; PRODUCTION SIZING FROM MEASUREMENT



The workstations are a development/VST resource, not a proposed SURF production-server specification.

## Inputs still needed

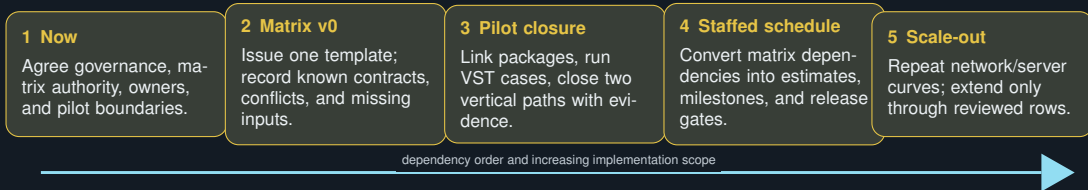
- ▶ SURF Ignition server, redundancy, storage, and virtualization specs.
- ▶ Controls network and managed-switch zones, routes, timing, and endpoint count.
- ▶ PLC/DPS platform, representative endpoints, and safe test-I/O boundary.
- ▶ Approved power/UPS/recovery cases and quantitative sizing criteria.

## Architecture rule

Gateway, bridge-process, host, and failover counts are separate sizing decisions.

# Move quickly in the right order

## A DISCUSSION STARTING POINT—DATES FOLLOW OWNERSHIP AND STAFFING



### Why this is faster

Unknowns are assigned early; integration work is ordered by dependency; package changes receive immediate compatibility evidence.

### Why this is safer

Unsupported guarantees and software-independent protection paths become visible before commissioning, not during recovery from a failed test.

The first deliverable is not more disconnected code: it is matrix v0 with owners, sources, and open questions.

# Decisions requested from this group

---

## AUTHORIZE THE SHARED VERSIONED WORKSPACE, THEN THE WORK

### Authorize Gate 0

1. **Create a collaboration-controlled SC/DPS Git group** with DAQ Git-admin support and named maintainers.
2. Use the interaction matrix to set technical priorities and schedule.
3. Name subsystem and lifecycle interface owners.
4. Approve two initial vertical slices and a crossing-fault case.

### Provide the missing inputs

1. Consortium variables, commands, constraints, interlocks, and recovery contracts.
2. DAQ Git-admin help creating the group and defining protections, releases, and retained CI evidence.
3. SURF VST and production Ignition/server/network specifications.
4. PLC, switch, PDU, power, UPS, and VST endpoint availability.

**Gate 0: create the shared Git group and publish matrix v0.**

**That turns the simulator from isolated work into a versioned integration and verification framework—and makes labor and completion estimates credible.**

# Appendix: current quantitative evidence

## LOCAL TEST BENCH AND SOURCE-DERIVED PLANNING VALUES

| Result              | Value                        | Evidence class                  | Boundary                                                                    |
|---------------------|------------------------------|---------------------------------|-----------------------------------------------------------------------------|
| Logical fleet       | 1,818 objects                | implemented + synthetic         | 190 DAPHNE, 480 WIB, 108 PDU, 208 power channels, sensors, permits, alarms. |
| OPC-UA namespace    | 40,755 variables             | workstation test                | One shared fleet endpoint; local Linux host.                                |
| Balanced campaign   | 12,000 attempts, 0 errors    | workstation test                | 3,000 trials/class; simulator time scale zero.                              |
| Bulk OPC-UA read    | 0.516 s                      | workstation test                | Software snapshot, not a real detector scan guarantee.                      |
| OPC-UA process knee | 4 processes                  | workstation test                | Same physical host; read and address-space churn workload.                  |
| DAPHNE floor        | 2.6901 s/board               | source-derived projection       | Explicit waits; some hardware programming time remains unknown.             |
| WIB floor           | 16.01–16.10 s/board          | source-derived projection       | COLDADC calibration/SPI waits; other hardware I/O remains unknown.          |
| WIB fleet candidate | 4 × 48 leaves, 48.30 s floor | posterior-predictive projection | Four bridge queues, three waves; requires async configuration jobs.         |

Primary source: Far-Detector Rack-Scale Slow-Control Emulation and Interface Verification, DCE-FD2VD-2026-07-27.

# Appendix: terms that must not be conflated

---

## ONE WORD—"SERVER"—HAD BEEN HIDING FOUR DIFFERENT DECISIONS

### Ignition Gateway

The SCADA application layer: tags, alarms, history, sessions, and operator views. One Gateway is the current development assumption.

### Bridge instance

One OPC-UA endpoint/process translating native device protocols into a fleet namespace. Four is the same-host monitoring candidate.

### Leaf worker

One active in-flight board configuration job. Forty-eight per bridge is the initial WIB planning point; it is not yet implemented.

### Physical host

A Linux machine or VM carrying one or more processes. Host and failover counts require a multi-host, representative-network test.

**Four bridge processes on one workstation is neither four Ignition Gateways nor four physical servers.**

# Appendix: interaction-matrix starter row

THE FORMAT SHOULD MAKE AN UNKNOWN IMPOSSIBLE TO HIDE

| Interaction       | Stage / state                        | Authority                            | Precondition                                        | Timing                                   | Failure / fallback                                           | Owner                                               | Evidence / status                                                                                 |
|-------------------|--------------------------------------|--------------------------------------|-----------------------------------------------------|------------------------------------------|--------------------------------------------------------------|-----------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Configure one WIB | commissioning; powered; timing ready | DAQ request through guarded bridge   | DPS permit; identity/config hash; FEMB power        | 16.10 s explicit-wait floor; timeout TBD | reject if permit lost; retry class TBD; independent readback | WIB / DAQ / SC / DPS owners to confirm              | Serializer mapped; simulator exercised; full OPC-UA job queue and real hardware timing unresolved |
| Rack power loss   | all stages; abnormal power           | independent protection path required | approved shut-down ordering and hold-up assumptions | APA-bias/WIB ordering bound TBD          | software may disappear; protection must remain effective     | electrical, APA-bias, WIB, SC/DPS owners to confirm | Conditional risk identified; physical path, timing, and proof test unresolved                     |

Rows begin as questions. They become schedule inputs only when the owner, source revision, acceptance criterion, and evidence route are explicit.

# Appendix: compare like with like

## QUASAR WAS THE BRIDGE; THE LARGE LOAD INCLUDED EMULATORS AND IGNITION TAGS

|                    | March 2026 WIB/Quasar test                                                           | July 2026 local fleet bench                                                                |
|--------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Runtime shape      | Many Python WIB emulator processes/endpoints → ZMQ → Quasar                          | 1,818 in-process logical objects → shared async runtime.                                   |
| Namespace / tags   | At 1,000 WIBs: ~ 5.5 million Ignition tags; only a small subset were OPC tags.       | 40,755 OPC-UA variables across the full fleet; compact read-only Ignition resources.       |
| Observed resources | At 480 WIBs: emulator 21.7 GB; Quasar 3.23 GB; Ignition 55.6 GB and ~ 6.7 CPU cores. | Emulator 0.65 GiB; DAPHNE bridge 0.05 GiB; Ignition 1.6 GiB.                               |
| Test boundary      | Large connection and Ignition tag/alarm stress test.                                 | Loopback interface test; production scan classes, history, and client load not reproduced. |

### Controlled comparison: hold the workload fixed and vary one layer

|                                                                                       |                                                                                                                                            |
|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Hold constant                                                                         | 480 identities and protobuf; OPC-UA nodes; Ignition UDTs/tags/alarms; scan, history, clients, and fault script.                            |
| Experiment A                                                                          | Same bridge and Ignition project; vary process-per-WIB versus shared-async emulation.                                                      |
| Experiment B                                                                          | Same emulator endpoints and Ignition project; vary Quasar versus the local bridge.                                                         |
| Measure                                                                               | Per-component CPU/RSS, descriptors, connections, cold start, throughput, p95/p99 latency, tag age/quality, errors, and reconnect recovery. |
| Only then compare efficiency against agreed latency, recovery, and resource headroom. |                                                                                                                                            |

Sources: DUNE Slow Controls ZMQ/OPC-UA Spin Test, 12 March 2026; DCE-FD2VD-2026-07-27.